

SaanSLive

See through the smog.

AI-Powered Urban Air Quality Intelligence Platform

ET AI Hackathon 2026 | Problem Statement 5: Smart Cities / Environmental Intelligence

Live: <https://saanslive.vercel.app>

Repository: <https://github.com/SyedArmanAli2003/SaanSLive>

Project Documentation

Detailed technical and product overview, prepared for hackathon submission.

1. Problem Statement

India's air quality crisis extends far beyond Delhi. Delhi averaged an AQI of 218 in 2024-25, classified 'Poor' or worse for over 200 days. Mumbai recorded dangerous AQI levels on 60+ days in 2024, and 24 of India's 50 most polluted cities are Tier 1 or Tier 2 urban centers, not just metros. The Lancet Planetary Health journal estimates 1.67 million premature deaths annually in India from air pollution.

India has deployed 900+ Continuous Ambient Air Quality Monitoring Stations under the National Clean Air Programme, yet a 2024 CAG audit found only 31% of cities with monitoring data have any actionable response protocol tied to it. The data exists; the intelligence layer to act on it does not. What's missing is not more monitoring — it's hyperlocal forecasting, source-level attribution, and a way to turn a raw number into a decision a person or a city administrator can actually act on, before the day happens.

2. Our Solution

SaanSLive is a hyperlocal air quality forecasting platform that predicts AQI several hours ahead per monitoring station, benchmarks every forecast against an honest naive baseline, and translates the result into a plain-language health advisory — both as a static panel and through a live, tool-calling AI assistant grounded in real data. Beyond individual forecasts, the platform surfaces a national Hotspot Prioritization view and a Compare Cities dashboard, taking the first step toward the brief's city-administrator-facing goals using only data that is genuinely available today.

Core design principle: never show a number the system cannot back up.

Throughout the product, missing data is shown honestly ("no forecast available yet", "forecast pending") rather than estimated or fabricated. Every disclaimer in the UI (e.g. on the Hotspot panel) is load-bearing, not decorative — it states exactly what the ranking is and is not based on.

3. Key Features

- **Hyperlocal AQI Forecasting** — Per-station XGBoost models forecast AQI ahead in time, using lag features, rolling averages, weather data, and time-of-day/day-of-week patterns. Every forecast is benchmarked against a persistence baseline ("tomorrow = today") and reports the real RMSE improvement or shortfall — honestly, per station.
- **Personalized Health Advisory** — A templated, plain-language advisory (e.g. category, station, time, and guidance for children/elderly/asthma) generates instantly. An optional LLM-polish layer (NVIDIA NIM, minimax-m3 default) rephrases it more naturally and can translate it to the user's preferred language, with the deterministic template as a guaranteed fallback if the API is unavailable.
- **Live AI Assistant (tool-calling)** — A floating conversational assistant, available on every page, answers questions about current AQI, forecasts, and city comparisons. Every factual claim is grounded in a real-time Supabase query via OpenAI-compatible function calling — never an invented number — and responses are tagged with which live data sources were checked.
- **Hotspot Prioritization** — Ranks every monitoring station nationally by urgency, combining current AQI severity with a real 7-day trend into a transparent, auditable priority score. A visible disclaimer clarifies the ranking reflects observed severity and trend only, not registered pollution-source data.
- **Compare Cities** — A national at-a-glance view of current vs. forecasted AQI across every covered city, with cities lacking a trained model honestly marked "forecast pending" rather than estimated.

- **Interactive Map + Onboarding** — A live map colour-codes every station by AQI severity (US EPA bands). A one-time onboarding flow captures vulnerability flags (children/elderly/asthma) and language preference to personalise advisories — stored only in browser localStorage, never on a server.
- **Geolocation Auto-Detect** — On first load, the dashboard optionally uses the browser's geolocation to default to the user's nearest station, falling back silently and gracefully if permission is denied.

4. Technical Architecture

The system is built entirely on free-tier infrastructure and runs fully automated, unattended, with no paid servers.

4.1 Data & Automation Pipeline

Stage	What it does
Data sources	OpenAQ (AQI / PM2.5 readings) and Open-Meteo (hourly weather) — both free, no ongoing cost.
Automated pipeline	A GitHub Actions cron job runs the ingestion + forecasting pipeline on a schedule, fully unattended: station sync, readings ingestion, weather ingestion, and forecast prediction, each fault-isolated so one step failing doesn't block the others.
Forecasting model	Per-station XGBoost models (trained offline as data accumulates), using time-based train/test splits to avoid data leakage, feature engineering via merge_asof-based lag alignment for irregular sensor intervals, and an explicit feasibility gate that refuses to train/predict when there isn't enough real history.
Database	Supabase Postgres — five core tables (stations, readings, weather, forecasts, user_profiles), idempotent upserts throughout (ON CONFLICT patterns) so re-running ingestion never creates duplicates, and Row Level Security enabled on every table.
Frontend	Next.js 14 (App Router) + TypeScript + Tailwind CSS, deployed on Vercel. Reads live data directly from Supabase; no custom backend server required for reads.

4.2 Tech Stack Summary

Layer	Technology
Frontend	Next.js 14, TypeScript, Tailwind CSS, Recharts, Leaflet/React-Leaflet
Backend / ML	Python, pandas, XGBoost, scikit-learn, SQLAlchemy (engine-only, no ORM models)
Database	Supabase (Postgres), Row Level Security, Supabase Anonymous Auth (evaluated, later replaced by local-only preferences for privacy)
Automation	GitHub Actions (scheduled + manual dispatch)
AI / LLM	NVIDIA NIM (minimax-m3 default, with gpt-oss-120b and deepseek-v4-flash as selectable alternatives, chosen via real benchmarked latency/reliability testing)
Hosting	Vercel (frontend), Supabase (database) — both free-tier

4.3 Privacy by Design

User personalization (vulnerability flags, preferred language, preferred station) is stored exclusively in browser localStorage. This was a deliberate architectural decision: an earlier iteration used Supabase Anonymous Authentication, which was removed once it became clear it created a permanent server-side identity record for every visitor — an unnecessary liability for a health-adjacent product asking about children, elderly, and asthma status. No login, no account, no persistent record of who visits

5. Model Performance (Honest Results)

Every forecast is benchmarked against a persistence baseline ("AQI stays the same as it currently is"). This is deliberately the hardest fair test available, and results are reported for every trainable city, including the ones where the model does not yet outperform the baseline.

City	XGB RMSE vs. Baseline	Improvement
Surat	25.34 vs 39.89	+36.5%
Guwahati	30.13 vs 44.12	+31.7%
Bhopal	39.35 vs 56.94	+30.9%
Mumbai	23.40 vs 32.55	+28.1%
Ahmedabad	33.65 vs 46.19	+27.1%
Delhi	29.99 vs 39.20	+23.5%
Pune	12.32 vs 15.00	+17.9%
Kolkata	19.41 vs 22.59	+14.1%
Chandigarh	31.06 vs 36.14	+14.1%
Kanpur	22.07 vs 24.68	+10.6%
Jaipur	37.01 vs 41.21	+10.2%
Chennai	15.29 vs 16.76	+8.8%
Bengaluru	10.72 vs 11.71	+8.4%
Nagpur	35.83 vs 35.82	≈0% (tie)
Patna	24.33 vs 21.93	−10.9% (baseline wins)
Indore	26.59 vs 19.20	−38.5% (baseline wins)
Hyderabad	25.96 vs 14.97	−73.4% (baseline wins)
Lucknow	50.80 vs 27.73	−83.2% (baseline wins)

13 of 18 trainable cities beat the naive baseline. Kochi and Visakhapatnam are excluded entirely: both have zero ingested AQI readings to date (a data-source availability gap, not a modelling limitation), and are honestly reported as such rather than silently omitted. Where the model underperforms baseline (Patna, Indore, Hyderabad, Lucknow), this is disclosed rather than hidden — a defensible, honest evaluation practice.

6. Live System Snapshot

Metric	Value
Monitoring stations tracked	53 stations across 20 cities
Historical AQI readings ingested	47,310+
Historical weather records ingested	17,099+

Metric	Value
Active forecast rows (live)	145+ (accumulating every automation cycle)
Cities with at least one trained model	18 of 20

Figures reflect the live production database at time of writing and grow automatically with each scheduled pipeline run.

7. Known Limitations (Stated Honestly)

- Kochi and Visakhapatnam have zero ingested AQI readings from available public sensors and therefore cannot be forecast until a working data source exists for them.
- Geospatial pollution-source attribution (an example feature in the original brief) is intentionally not built: it requires satellite imagery and registered-emitter data this team does not have reliable access to. Rather than fabricate a source-attribution claim, the Hotspot Prioritization feature ranks by observed severity and trend only, with this boundary explicitly disclosed in the UI.
- Four of eighteen trainable cities (Patna, Indore, Hyderabad, Lucknow) do not yet outperform the naive persistence baseline, most likely due to limited historical data volume at time of training; expected to improve as more data accumulates.
- The system has not yet been load-tested at scale; current usage patterns are hackathon-demo scale.

8. Alignment with Judging Criteria

Criterion	How SaanSLive addresses it
Innovation (25%)	A live, tool-calling AI assistant grounded in real-time database queries (not a static wrapper); an honestly-disclaimed Hotspot Prioritization signal; privacy-first, account-free personalization.
Business Impact (25%)	Directly serves the populations most able to act on a warning (parents, elderly residents, those managing respiratory conditions) with an advisory they can actually use, not just a number.
Technical Excellence (20%)	A real trained model benchmarked against an honest baseline, with per-city results reported transparently, including where it does not yet win; leakage-safe time-based splits; idempotent, fault-isolated automation.
Scalability (15%)	100% free-tier infrastructure; the same automated pipeline extends to any city with OpenAQ sensor coverage with no per-city engineering effort.
User Experience (15%)	A single, coherent dashboard (map, forecast, advisory, prioritization, comparison) plus a conversational assistant; honest empty/pending states throughout rather than broken or misleading UI.

9. Roadmap

- Expand multilingual advisory coverage using Indic-language-specialist models.
- Add real geospatial source-attribution once reliable satellite/land-use/registered-emitter data sources are identified.
- Close remaining data gaps (Kochi, Visakhapatnam) as new sensor sources become available.
- Extend city coverage as OpenAQ sensor availability grows, using the existing zero-additional-engineering pipeline.

PROJECT FILE STRUCTURE

```
ET-Hackathon/
├── schema.sql          # Supabase Postgres schema (5 tables + RLS)
├── report.md           # Full technical report & QA log
├── kiro.md             # Session-by-session build log (Kiro agent)
├── README.md          # This file
├── start.bat          # Windows one-click dev launcher
├── .github/workflows/
│   └── ingest.yml      # Cron every 5h + manual dispatch
├── ingestion/
│   ├── config.py       # 20 tracked Indian cities with lat/lng
│   ├── db.py           # SQLAlchemy singleton engine
│   ├── setup_stations.py # OpenAQ → stations (ON CONFLICT DO UPDATE)
│   ├── ingest_readings.py # OpenAQ sensors → readings (ON CONFLICT DO NOTHING)
│   ├── ingest_weather.py # Open-Meteo → weather (ON CONFLICT DO NOTHING)
│   ├── run_ingestion.py # Master orchestrator, fault-isolated, exact counts
│   ├── explore_data.py  # EDA: console report + AQI PNG plot
│   └── requirements.txt # Locked Python deps
├── model/
│   ├── features.py      # build_features / add_forecast_target / forecast_feasibility
│   ├── split.py         # time_split(df, test_days=2) → train/test/meta (no leakage)
│   ├── train.py         # Per-city XGBoost/LightGBM training + persistence-baseline eval
│   ├── predict.py       # Load artifacts, run inference, write to forecasts
│   └── artifacts/       # Trained .pkl files — one per city × model type × horizon
├── supabase/
│   ├── config.toml      # Supabase CLI project config
│   └── migrations/      # Schema + Postgres function migrations (source of truth)
├── frontend/saanslive/
│   ├── .env.local       # NEXT_PUBLIC_SUPABASE_URL + ANON_KEY (gitignored)
│   └── lib/
│       ├── data.ts      # Data layer — the ONLY file that queries Supabase directly
│       │               # getStations, getCurrentReading, getLatestForecasts,
│       │               # getHotspotRanking, getCityComparison
│       ├── aqi.ts       # AQI band mapping — single source of truth for all UI
│       ├── chatTools.ts # Real Supabase-backed tool implementations for the chatbot
│       ├── generateAdvisory.ts # Client-side template + calls /api/advisory to polish it
│       ├── nimModels.ts  # NVIDIA NIM model registry, default selection, settings
│       ├── geolocation.ts # Browser geolocation → nearest-station lookup
│       ├── localPreferences.ts # Per-device onboarding preferences (localStorage)
│       └── supabaseClient.ts # Single shared Supabase client instance
│       └── components/
│           ├── HeroSection.tsx # Landing hero (canvas-free cursor-reveal effect)
│           ├── StationMap.tsx  # Leaflet map with live AQI markers
│           ├── ForecastChart.tsx # Recharts 24h forecast + persistence baseline
│           ├── AdvisoryPanel.tsx # AI-polished health advisory with template fallback
│           ├── HotspotPanel.tsx # "Hotspot Prioritization" ranked table + disclaimer
│           ├── CityComparisonView.tsx # "Compare Cities" sortable table / bar chart toggle
│           ├── AqiChatbot.tsx  # Floating tool-calling AI chatbot
│           ├── OnboardingModal.tsx # First-visit personalization modal
│           └── Skeleton.tsx    # Shared loading placeholders
│       └── app/
│           ├── page.tsx      # Homepage — hero only
│           ├── about/page.tsx # About page
│           └── dashboard/page.tsx # Dashboard: Overview / Hotspot Prioritization / Compare Cities tabs
│           └── api/
│               ├── advisory/route.ts # Server-side LLM cascade for advisory rephrasing
│               └── chat/route.ts    # Server-side tool-calling loop for the AI chatbot
```